Intro to DL4MSc: Quantization

Alexander Binder

January 5, 2026

Outline

Quantization - Goials

goals:

- use less memory for inference
- use less inference time or inference compute
  - save energy = money during inference
  - embedded devices (drones) with limited mem, compute and battery capacity
- tradeoff: goal of low loss of accuracy compared to original weights

Weight quantization:

- map FP32 weights to floating point weights with less space usage, eg FP16 or FP8, or to integer weights like INT8 or UINT8

Activation quantization:

- map FP32 activations to floating point weights with less space usage, eg FP16 or FP8, or to integer values like INT8 or UINT8

Learning goals

- explain QAT vs PTQ and the basic approach to QAT
- explain the basic symmetric and asymmetric quantization for PTQ
- explain the differences between dynamic and static activation quantization
- explain the simple baseline for fitting the parameters for asymmetric and symmetric (max-abs) quantization
- explain the idea of the parametrization of WX before quantization in Xiao et al. 2022

for some nice graphics on FP32/16 types:
https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

some papers:

- Sec. 3 in Wu et al. https://arxiv.org/pdf/2004.09602
- Nagel et al. https://arxiv.org/abs/2106.08295
- Xiao et al. https://arxiv.org/pdf/2211.10438
- Lin et al. https://arxiv.org/pdf/2306.00978

some more difficult papers for bonus reading:

- Frantar et al. https://arxiv.org/abs/2210.17323 (difficult paper)
- Ashkboos et al. https://arxiv.org/pdf/2404.00456 (difficult paper)

The basics: Several ways to quantize. Here: affine schemes.

- asymmetric quantization to unsigned integers

$$x_{int} = q(x|s,z,b) = clamp(round(x/s) + z, [0, 2^b - 1]) \in \{0, \dots, 2^b - 1\}$$
$$\hat{x} = deq(x_{int}|s,z) = s(x_{int} - z) \text{ dequantization}$$

  - two trainable parameters s scale, zero point z
  - quantization limits: $q_{min} = s(0 - z)$, $q_{max} = s(2^b - 1 - z)$
  - abs. rounding error: $|x - \hat{x}| \leq 0.5s$

- symmetric quantization to signed integers:

$$x_{int} = q(x|s,b,-) = clamp(round(x/s), [-2^{b-1}, 2^{b-1} - 1]),\ z = 0$$
$$\hat{x} = deq(x_{int}|s) = sx_{int} \text{ dequantization}$$

  - $z = 0$, version here for signed integers. For non-neg maps (ReLU) one could use a version with unsigned integers

next: quantization-aware training (QAT) vs post training quantization (PTQ):

quantization-aware training (QAT)

(+) easy to implement

(+) potentially better performance

(–)!!! often infeasible for large LLMs (training costs)

(-) fine-tuning-based variant usually needs some additional PTQ-based mitigations

post training quantization (PTQ)

(+) the vanilla option with a given pretrained model

(-) more elaborate to implement to deal with problems

(-) potentially worse performance compare to quantization-aware training

quantization-aware training (QAT):

very simple idea:

- simulate effects of quantization during pretraining of the LLM

quantization-aware training (QAT):

very simple idea:

- for every weight to be quantized apply in the forward pass a quantization followed by dequantization
- for every activation to be quantizated do the same in the forward pass

$$\hat{w} = deq(q(w|s,z,b)|s,z,b)$$
$$\hat{a} = deq(q(a|s,z,b)|s,z,b)$$

  - simulates during training the rounding errors of quantization
- backward pass / gradients ? use straight-through-estimator:

$$\frac{d\hat{v}}{dv}(v) = \begin{cases} 1 & \text{if } v \in [q_{min}, q_{max}] \\ 0 & \text{if } v \notin [q_{min}, q_{max}] \end{cases}$$

(quantization is not differentiable ...)

if Batch-normalization folding is done at inference time, simulate this, as well:

- See section 3.2 in https://openaccess.thecvf.com/content_cvpr_2018/papers/Jacob_Quantization_and_Training_CVPR_2018_paper.pdf
- simulate the fusion of the Batch normalization into the next convolution layer

QAT in code: https://pytorch.org/blog/quantization-aware-training/

next: flavours of post training quantization (PTQ):

- quantize weights only
- weight quantization and dynamic activation quantization
- weight quantization and static activation quantization

next: flavours of post training quantization (PTQ):

- quantize weights only
  - memory reduction due to less weights
  - might lead to more efficiency if less GPUs are used (e.g. 70B params in FP32 $\approx 280GB$)
  - speedups only possible with custom implementations of mixed-precision kernels for multiplications of int8 with fp32 or fp16, see eg Fig 9 in Lin et al. https://arxiv.org/pdf/2306.00978 for Ours(FP16) vs Ours(W4A16)

- common alternative: weight quantization and dynamic activation quantization
  - quantize activations to the same type as the weights on-the-fly. Therefore can use standard kernels for low FP or for INT8.
  - compute activation quantization parameters (s, z) for every input sample and every channel
- (+) no activation outliers
- (-) on the fly operations lead to compute overhead
- (-) reading FP32 activations can lead to higher mem bandwith compared to pure INT8 pipelines where activations are statically quantized!

next: flavours of post training quantization (PTQ):

- ⊙ weight quantization and static activation quantization
  - · compute activation quantization parameters ((s, z) per layer) using a separate calibration dataset
  - (-) need to deal with activations at inference time outside $[q_{min}, q_{max}]$ which were not seen in calibration samples. Thus often more complicated approaches
  - (+) can lead to inference speedups due to lower memory bandwidth requirements

A simple baseline for quantization:
https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

- asymmetric: fit (s, z). Given a set of activations $\{a\}$ or weights $\{w\}$, map to $[0, 2^b - 1]$
- Idea: fit min value to 0, fit max value to $2^b - 1$.
  We know that

$$a \mapsto \frac{a - \min(\{a\})}{\max(\{a\}) - \min(\{a\})} \in [0, 1]$$

  Now need to multiply this only by $2^b - 1$, then separate it into an a-dependent term minus the offset: $a/s - z$

Try affine mapping of activations without rounding:

$$
\begin{aligned}
a \mapsto & \frac{a - \min(\{a\})}{\max(\{a\}) - \min(\{a\})} * (2^b - 1) \\
= & \, a \frac{2^b - 1}{\max(\{a\}) - \min(\{a\})} - \frac{\min(\{a\})(2^b - 1)}{\max(\{a\}) - \min(\{a\})}
\end{aligned}
$$

next: write this as $a/s - z$:

$$
\begin{aligned}
& = a/(\frac{\max(\{a\}) - \min(\{a\})}{2^b - 1}) - \min(\{a\})/(\frac{\max(\{a\}) - \min(\{a\})}{2^b - 1}) \\
& = a/s - \min(\{a\})/s \\
\Rightarrow s & = \frac{\max(\{a\}) - \min(\{a\})}{2^b - 1} \\
\Rightarrow z & = -round(\min(\{a\})/s)
\end{aligned}
$$

A simple baseline for quantization:
https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

- asymmetric: fit (s, z). Given a set of activations $\{a\}$ or weights $\{w\}$, map to $[0, 2^b - 1]$

$$\Rightarrow s = \frac{\max(\{a\}) - \min(\{a\})}{2^b - 1}$$
$$\Rightarrow z = -round(\min(\{a\})/s)$$

If one would want to fit not in $[0, 2^b - 1]$ but into $[-2^{b-1}, 2^{b-1} - 1]$, then $z_{sym} = z - 2^{b-1}$

because $[0, 2^b - 1] - 2^{b-1} = [-2^{b-1}, 2^{b-1} - 1]$

A simple baseline for quantization:
https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

- symmetric: fit s only. Given a set of activations $\{a\}$ or weights $\{w\}$, map to $[-2^{b-1}, 2^{b-1}-1]$
- therefore $\max(\{a\}) \mapsto v < 2^{b-1}-1$, $\min(\{a\}) \mapsto v > -2^{b-1}$

  a simple solution is dividing by the maximum absolute value to get something bounded in $[-1,+1]$, after that multiply by 2^{b-1}:

$$a \mapsto \frac{a}{\underbrace{\max(\{|a|\})}_{\in[-1,+1]}} \mapsto \frac{a}{\max(\{|a|\})} * (2^{b-1}-1)$$

$$\Rightarrow s = \max(\{|a|\})/(2^{b-1}-1)$$

Symmetric or asymmetric? Consider dequantization step to compute activation times weight:

$$\begin{aligned} deq(w)deq(x) &= s_w(w_{int} - z_w)s_x(x_{int} - z_{int}) = s_w s_x(w_{int} - z_w)(x_{int} - z_{int}) \\ &= s_w s_x w_{int} x_{int} - s_w s_x x_{int} z_w - s_w s_x z_{int} w_{int} + s_w s_x z_{int} z_w \end{aligned}$$

- first term: data-dependent, present if all symmetric
- terms in blue: data-independent, can be precomputed once
- last term in red: data-dependent, can be dropped if weight quantization is done symmetrically. For this reason often weight quantization is done symmetrically

baseline for quantization:

- use the asymmetric variant for activations, get *max*/*min*-values either on the fly (for weights + dynamic) or compute *max*/*min*-values on activations on calibration data
- round weights to nearest after quantization
- often big loss in accuracy with this, eg Tables 3 and 4 in Frantar et al. https://arxiv.org/pdf/2210.17323, see also the RTN baseline results in Lin et al. https://arxiv.org/pdf/2306.00978
- next: some better approaches

a well readable paper with an improved method: Xiao et al. https://arxiv.org/pdf/2211.10438

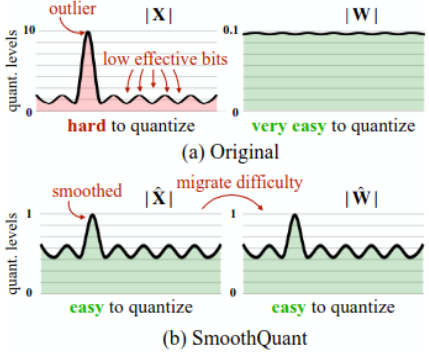

Figure 2: SmoothQuant's intuition: the activation $\mathbf{X}$ is hard to quantize because outliers stretch the quantization range, leaving few effective bits for most values. We migrate the scale variance from activations to weights $\mathbf{W}$ during offline to reduce the quantization difficulty of activations. The smoothed activation $\hat{\mathbf{X}}$ and the adjusted weight $\hat{\mathbf{W}}$ are both easy to quantize.

a well readable paper with an improved method: Xiao et al. https://arxiv.org/pdf/2211.10438

- see Section 3
- see "However, per-channel activation quantization does not map well to hardware-accelerated GEMM kernels" on p3, right column
- uses W8A8 means INT8 for both
- Fig 8 for speedups

a well readable paper with an improved method: Xiao et al. https://arxiv.org/pdf/2211.10438
idea:

- per sample activations have bigger scale difference issues than weights
- downscale activations and upscale weights before quantization:

$$Y = XW = (Xdiag(s^{-1}))(Wdiag(s))$$

$$(Xdiag(s^{-1}))(Wdiag(s)) \mapsto \widehat{X}\widehat{W}$$

The last equation say: quantize the scaled weights and activations **after** s was determined.

- need to find s by dynamic or static approaches

- before the actual quantization step: optimize on a set of calibration samples the tradeoff hyperparameter α, per-tensor (or, slower, per-token), see Table 2 in the paper:

$$s_j = \max |X_j|^{\alpha} / max(|W_j|)^{1-\alpha}$$

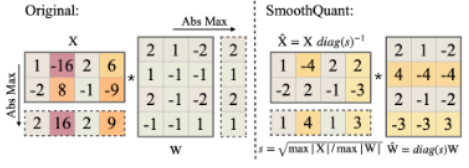

Figure 5: Main idea of SmoothQuant when α is 0.5. The smoothing factor s is obtained on calibration samples and the entire transformation is performed offline. At runtime, the activations are smooth without scaling.

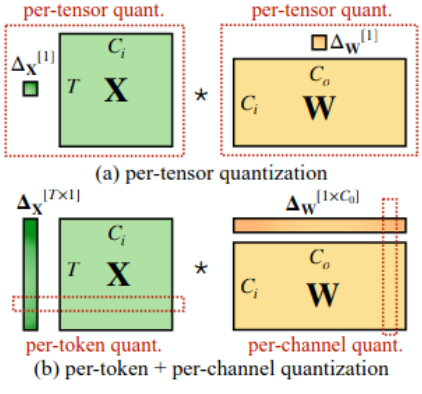

Figure 3: Definition of per-tensor, per-token, and per-channel quantization. Per-tensor quantization is the most efficient to implement. For vector-wise quantization to efficiently utilize the INT8 `GEMM` kernels, we can only use scaling factors from the outer dimensions (i.e., token dimension T and out channel dimension C_o) but not inner dimension (i.e., in channel dimension C_i).

(out of exams):

The AWQ paper https://arxiv.org/pdf/2306.00978 builds on a similar idea but with asymmetric data types for weights and activations – see section 4 there.

Further idea if quantization yields bad accuracy: **Selective quantization**

- quantize single layer, measure impact on accuracy, loop ove all layers
- obtain an order for best-quantizable layers using this greedy estimate
- quantize K best quantizable layers by dropping the most sensitive ones

- LoRA and variants for better memory efficiency during finetuning
- quantization for memory and compute efficiency during inference